

INFORME DE MÉTRICAS

Diseño Sostenible y Principios SOLID

Caso: DentaCare — Sistema legado de citas odontológicas (C#)

Arquitectura de Software I

Entregable 1

Juan José Medina Sepúlveda
Manuel Alejandro Domínguez Guerrero

Institución Universitaria ITM
Medellín, Antioquia
07/10/2026

1. Introducción y alcance

Este informe presenta el diagnóstico del sistema legado **DentaCare.Legacy** (C#, citas odontológicas) y su refactorización **DentalCare.Refactored**: violaciones SOLID, métricas de cohesión y acoplamiento antes y después, y decisiones de arquitectura (ADRs, sección 9). El código es simulado: no hay base de datos, SMTP ni Twilio reales.

1.1. Organización del informe

Fase de la práctica	Sección	Contenido
Fase 1 — Matriz de violaciones SOLID	4 y 5	Violaciones de SRP, OCP, LSP, ISP y DIP con evidencia por archivo y línea, más hallazgos adicionales.
Fase 1 — Línea base cuantitativa	6	LCOM96b, Ca, Ce, I, A, D, Zona de Dolor y complejidad ciclomática del legado.
Fase 2 — Refactorización	7	Estructura de la solución, trazabilidad con el legado, solución por principio y verificación funcional.
Fase 3 — Recálculo de métricas	8	Métricas por clase y por paquete, tabla comparativa y efecto en los atributos de calidad.
Fase 4 — ADRs	9	ADR-001, ADR-002, ADR-003, ADR-004 y ADR-005, ubicados en docs/adr/.

1.2. Convenciones

Las líneas se citan como Archivo.cs, L.n (la directiva using inicial es la L.1). Las métricas se calcularon de forma manual con las definiciones del material de clase.

Métrica	Fórmula	Lectura
LCOM96b	$LCOM96b = (1 / a) \times \sum [(m - \mu(A_j)) / m]$	0,0 es cohesión perfecta; 1,0 es ausencia de cohesión. a: atributos de instancia; m: métodos; $\mu(A_j)$: métodos que usan el atributo A_j .
Inestabilidad	$I = Ce / (Ce + Ca)$	0,0 es máxima estabilidad ($Ca > 0$, $Ce = 0$); 1,0 es máxima inestabilidad ($Ca = 0$, $Ce > 0$).
Abstracción	$A = \sum m_a / \sum m_c$	Proporción de elementos abstractos (interfaces y clases abstractas) sobre el total de tipos. 0,0 es un módulo totalmente concreto; 1,0 uno totalmente abstracto.
Distancia	$ D = A + I - 1 $	Separación respecto de la Secuencia Principal. Cuanto más cerca de 0, más equilibrada es la relación entre abstracción e inestabilidad.

2. Descripción del sistema legado

DentaCare.Legacy tiene siete archivos; la lógica y la orquestación se concentran en GestorCitasOdontologicas, que instancia directamente su infraestructura.

Archivo	Tipo	Rol
Program.cs	Punto de entrada	Crea el gestor con new y ejecuta tres flujos: agendar, cancelar y reportar totales.
GestorCitasOdontologicas.cs	Clase de servicio (God Class)	Disponibilidad, tarifas, creación de la cita, persistencia, notificación, cancelación y totales en memoria.
SqlServerEjecutor.cs	Infraestructura	Persistencia en SQL Server con SqlCommand embebido y cadena de conexión con credenciales.
NotificacionServicio.cs	Infraestructura	Envío de correo (SMTP) y SMS (Twilio simulado) en un único método.
Cita.cs	Entidad	Datos de la cita y GenerarTextoConfirmacion().
Paciente.cs	Entidad	Datos del paciente; TipoConvenio codificado como entero.
Odontologo.cs	Entidad	Datos del odontólogo; EspecialidadId codificado como entero.

2.1. Responsabilidades acumuladas en GestorCitasOdontologicas

El gestor mezcla ocho responsabilidades: disponibilidad (L.14–17), tarifa (L.19–37), descuentos y recargos (L.39–57), creación de la cita (L.58–66), persistencia (L.68, 88), notificaciones (L.69, 89), cancelación y penalización (L.77–86) y acumuladores de reporte (L.9–10, 71, 91, 95, 97).

3. Línea base funcional

Resultado esperado del escenario de Program.cs: paciente EPS en primera visita, odontólogo de Cirugía, radiografía requerida, cita a 12 horas y cancelación inmediata. Los valores 130,00 y 90,00 son los casos de aceptación de la versión refactorizada.

Paso	Regla aplicada (GestorCitasOdontologicas.cs)	Resultado
Costo base de consulta	L.19	100,00
Especialidad Cirugía (×2,5)	L.30–33	250,00
Convenio EPS (×0,30)	L.40–43	75,00
Primera vez (+20)	L.49–52	95,00
Radiografía (+35)	L.54–57	130,00 (copago final)
Cancelación a menos de 24 h: 50 + 40 por Cirugía	L.79–83	90,00 (penalización)

4. Fase 1 — Diagnóstico SOLID

Principio	Tipo de violación	Síntesis
SRP	Directa	GestorCitasOdontologicas reúne al menos ocho responsabilidades que responden a actores distintos: negocio, tesorería, infraestructura y comunicación.
OCP	Directa	Cadenas if/else por especialidad, convenio y tipo de cita: cada variante nueva obliga a modificar AgendarCita o CancelarCita.
LSP	Indirecta	Códigos enteros y cadenas sustituyen al polimorfismo; no existe un contrato que un subtipo pueda respetar o romper.
ISP	Indirecta	Contratos concretos y amplios: un solo método envía correo y SMS, y un solo servicio de datos agrupa todas las operaciones.
DIP	Directa	El módulo de alto nivel crea con new sus dependencias concretas, y las de infraestructura tienen credenciales incrustadas.

4.1. SRP — Single Responsibility Principle

Una clase cambia por razones de cuatro actores: clínica, finanzas, tecnología (SQL Server, SMTP, Twilio) y gerencia (reportes).

- GestorCitasOdontologicas.cs concentra las responsabilidades de 2.1; NotificacionServicio.cs, L.12–21, mezcla correo, SMS, configuración y credenciales; Cita.cs, L.16, genera texto de presentación.

4.2. OCP — Open/Closed Principle

- GestorCitasOdontologicas.cs: if / else if por especialidad (L.22–37), descuentos por convenio (L.40–47), recargos fijos (L.49–57) y penalización de 50 con recargo de 40 por Cirugía (L.79–83). NotificacionServicio.cs, L.14–21: un canal nuevo obliga a modificar EnviarEmailYSms.

4.3. LSP — Liskov Substitution Principle

Sin herencia ninguna subclase rompe su contrato; el problema es que no hay contrato: las variantes son códigos primitivos que el consumidor debe conocer uno a uno, síntoma equivalente al instanceof que en clase se señala como indicio de violación de LSP.

- Paciente.cs, L.13, y Odontologo.cs, L.11, usan int; Cita.cs, L.14, usa string; GestorCitasOdontologicas.cs, L.16, lanza una Exception genérica. SqlServerEjecutor y NotificacionServicio no implementan abstracción alguna, así que no hay sustituto posible.

4.4. ISP — Interface Segregation Principle

- NotificacionServicio.cs, L.12–21, obliga a enviar correo y SMS juntos y recibe la Cita completa. SqlServerEjecutor.cs, L.11 y L.25, mezcla agendamiento y cancelación. Program.cs depende de un gestor que combina comandos y consultas.

4.5. DIP — Dependency Inversion Principle

- GestorCitasOdontologicas.cs, L.7–8, crea sus dependencias con new. SqlServerEjecutor.cs, L.10, y NotificacionServicio.cs, L.10–11, incrustan credenciales de SQL Server, SMTP y Twilio. Consecuencia: AgendarCita no se puede probar sin base de datos ni servidor de correo, y cambiar de proveedor obliga a modificar la lógica de negocio.

5. Hallazgos adicionales

#	Hallazgo	Detalle
1	Secretos incrustados	Contraseña de SQL Server (SqlServerEjecutor.cs, L.10) y clave de Twilio (NotificacionServicio.cs, L.11). Es un requisito de seguridad que condiciona la arquitectura de configuración.
2	Parámetro mensaje ignorado	EnviarEmailYSms(cita, mensaje) no usa mensaje (L.12); el cuerpo está fijo en L.17. La notificación de cancelación (GestorCitasOdontologicas.cs, L.89) envía un correo que dice «cita programada».
3	using System.Data.SqlClient innecesario	GestorCitasOdontologicas.cs, L.1: la clase no usa SQL directamente, pero declara la dependencia.
4	Estado en memoria	_totalRecaudadoMes y _totalCitasCanceladas (L.9–10) se pierden al reiniciar, no son seguros ante concurrencia y el total recaudado no se reduce al cancelar.
5	Obsesión por primitivos	Especialidad, convenio y estado como int o string; identificador de cita con Guid.NewGuid().ToString().Substring(0, 8) (L.60), con riesgo de colisión.

6. Fase 1 — Línea base cuantitativa

6.1. Criterios de medición

- **m = 4** (AgendarCita, CancelarCita, ObtenerTotalRecaudado, ObtenerTotalCanceladas) y **a = 4** (_db, _notificador, _totalRecaudadoMes, _totalCitasCanceladas). Los tipos del framework se excluyen de Ca y Ce y se reportan como sensibilidad; A es la proporción de interfaces y clases abstractas sobre el total.

6.2. LCOM96b

$$\text{LCOM96b} = (1 / a) \times \Sigma [(m - \mu(A_j)) / m]$$

Atributo (A _j)	Métodos que lo acceden	μ(A _j)	(m – μ) / m
_db	AgendarCita (L.68), CancelarCita (L.88)	2	(4 – 2) / 4 = 0,50
_notificador	AgendarCita (L.69), CancelarCita (L.89)	2	(4 – 2) / 4 = 0,50
_totalRecaudadoMes	AgendarCita (L.71), ObtenerTotalRecaudado (L.95)	2	(4 – 2) / 4 = 0,50
_totalCitasCanceladas	CancelarCita (L.91), ObtenerTotalCanceladas (L.97)	2	(4 – 2) / 4 = 0,50
Σ			2,00

$$\text{LCOM96b} = (1 / 4) \times 2,00 = 0,50$$

Interpretación. 0,50 es cohesión intermedia: dos grupos de métodos, {AgendarCita, ObtenerTotalRecaudado} y {CancelarCita, ObtenerTotalCanceladas}, unidos solo por _db y _notificador. Como la mezcla ocurre dentro de AgendarCita, la métrica subestima el problema (ver Ce y 6.8).

6.3. Acoplamiento aferente (Ca) y eferente (Ce)

Ca = 1 (Program.cs, L.8). **Ce = 5:** SqlServerEjecutor, NotificacionServicio, Cita, Paciente y Odontologo; con los tipos del framework, Ce sería 9 e I, 0,90.

6.4. Inestabilidad (I)

$$I = Ce / (Ce + Ca) = 5 / (5 + 1) = 0,833$$

6.5. Abstracción (A)

$$\text{Gestor: } A = \Sigma m_a / \Sigma m_c = 0 / 1 = 0,00$$

$$\text{Solución: } A = 0 \text{ tipos abstractos} / 6 \text{ tipos en total} = 0,00$$

No hay interfaces ni clases abstractas en la solución; por miembros también $A = 0 / 8 = 0,00$.

6.6. Distancia a la Secuencia Principal (D)

$$|D| = |A + I - 1| = |0,00 + 0,833 - 1| = 0,167$$

6.7. Zona de Dolor (Zone of Pain)

El gestor **no cae en la Zona de Dolor** ($A \approx 0$ e $I \approx 0$), pues $I = 0,833$. Su problema son el Ce alto con dependencias solo concretas, la cohesión que oculta responsabilidades mezcladas y la falta de abstracciones. Paciente y Odontologo sí caen en la zona, tolerable por ser datos estables.

Tabla complementaria: demás tipos de la solución

Tipo	Ca	Ce	I	A	D	Observación
GestorCitasOdontologicas	1	5	0,83	0,00	0,17	Cerca de la Secuencia Principal; fuera de la Zona de Dolor.
SqlServerEjecutor	1	1	0,50	0,00	0,50	Concreto; detalle de infraestructura.
NotificacionServicio	1	1	0,50	0,00	0,50	Concreto; detalle de infraestructura.
Cita	4	2	0,33	0,00	0,67	Muy usada y concreta: tendencia hacia la Zona de Dolor.
Paciente	3	0	0,00	0,00	1,00	Zona de Dolor (entidad estable).
Odontologo	3	0	0,00	0,00	1,00	Zona de Dolor (entidad estable).

En los tipos sin métodos $A = 0$; el Ce de SqlServerEjecutor y NotificacionServicio cuenta solo Cita.

6.8. Métrica complementaria: complejidad ciclomática

Método	Puntos de decisión	Complejidad
AgendarCita	L.14, 22, 26, 30, 34, 40, 44, 49 y 54 (9 decisiones)	10
CancelarCita	L.79 y 81 (2 decisiones)	3
ObtenerTotalRecaudado	Sin decisiones	1
ObtenerTotalCanceladas	Sin decisiones	1

AgendarCita concentra casi toda la complejidad, lo que respalda el diagnóstico de God Class que LCOM96b no refleja.

7. Fase 2 — Refactorización y diseño sostenible

DentalCare.Refactored reemplaza la clase monolítica por componentes de una sola razón de cambio que se comunican por interfaces y se ensamblan en Program.cs, con adaptadores en memoria en lugar de SQL Server, correo y SMS.

7.1. Criterios de diseño adoptados

- Separar casos de uso, reglas de negocio y detalles técnicos, con cada variante de negocio como regla independiente de contrato común; servicios que dependen solo de abstracciones recibidas por constructor (*Pure DI*, Seemann y van Deursen, 2019), sin contenedor IoC; y enumeraciones, records y una excepción específica en lugar de códigos enteros y cadenas mágicas, conservando el comportamiento del legado (sección 3).

7.2. Organización de la solución

Carpeta (namespace)	Contenido	Responsabilidad
Entidades	Cita, Paciente, Odontologo	Datos del dominio. Cita expone propiedades de solo lectura y encapsula su cambio de estado en el método Cancelar.
Enumeraciones	Convenio, Especialidad, EstadoCita	Sustituyen los enteros (1, 2, 3...) y las cadenas («PROGRAMADA», «CANCELADA») del legado.
Excepciones	OdontologoNoDisponibleException	Error de negocio con tipo propio en lugar de Exception genérica.
Interfaces	15 contratos	Abstracciones de las que dependen los servicios (ver 7.4).
Modelos	SolicitudAgendamiento, ContextoCancelacion, MensajeNotificacion	Entidades y records que encapsulan la información de la operación.
Reglas/Tarifas	4 tarifas por especialidad, 2 descuentos por convenio y 2 recargos	Cálculo del copago, una regla por clase.
Reglas/Penalizaciones	PenalizacionCancelacionTardia, RecargoCancelacionTardiaCirugia	Política de cancelación, una regla por clase.
Reglas	PoliticaCancelacion	Constante del límite de 24 horas que comparten las reglas de penalización.
Servicios	12 clases	Casos de uso, calculadores que componen reglas, notificador compuesto y adaptadores en memoria.
(raíz)	Program.cs	Composition root: crea y conecta todas las dependencias.

7.3. Trazabilidad entre el legado y la solución refactorizada

Responsabilidad en el legado	Componente refactorizado	Principio
Validación de disponibilidad (L.14–17)	ValidadorDisponibilidadPorBandera : IValidadorDisponibilidad	SRP, DIP
Tarifa por especialidad (L.19–37)	TarifaOrtodoncia, TarifaEndodoncia, TarifaCirugia, TarifaOdontopediatria : IReglaTarifa	OCP
Descuentos por convenio (L.39–47)	DescuentoEps, DescuentoPrepagada : IReglaTarifa	OCP
Recargos de primera vez y radiografía (L.49–57)	RecargoPrimeraVez, RecargoRadiografia : IReglaTarifa	OCP
Composición del copago (L.19–57)	CalculadorCopago : ICalculadorCopago	SRP, OCP
Creación de la cita e identificador (L.58–66)	GuidGeneradorIdentificador : IGeneradorIdentificador y constructor de Cita	SRP, DIP
Persistencia en SQL Server (L.68, L.88; SqlServerEjecutor.cs)	RepositorioEnMemoria : ICitaRepositorio, ICancelacionRepositorio	DIP, ISP
Notificación por correo y SMS (L.69, L.89; NotificacionServicio.cs)	NotificadorCompuesto : INotificador, que delega en CanalCorreoEnMemoria y CanalSmsEnMemoria : ICanalNotificacion	DIP, ISP, OCP
Texto de los mensajes (Cita.cs, L.16; NotificacionServicio.cs, L.17)	ConstructorMensajesCita : IConstructorMensajes	SRP
Política de cancelación (L.77–83)	PenalizacionCancelacionTardia, RecargoCancelacionTardiaCirugia : IReglaPenalizacion, compuestas por CalculadorPenalizacion	SRP, OCP
Cambio de estado de la cita (L.85–86)	Cita.Cancelar(penalizacion)	SRP
Acumuladores de reporte (L.9–10, 71, 91, 95, 97)	ReporteCitasEnMemoria : IRegistroReportes, IConsultaReportes	SRP, ISP
Orquestación de agendar (L.12–73)	ServicioAgendamiento : IServicioAgendamiento	SRP, DIP
Orquestación de cancelar (L.75–93)	ServicioCancelacion : IServicioCancelacion	SRP, DIP

7.4. Contratos y flujo de dependencias

Las 15 interfaces son 2 de casos de uso, 5 de reglas de negocio, 5 puertos hacia infraestructura y 3 de soporte. Program.cs es el único lugar con new sobre implementaciones (composition root): ServicioAgendamiento (7 interfaces) y ServicioCancelacion (5) dependen solo de interfaces, y los

calculadores y el notificador compuesto reciben colecciones de reglas (8 de tarifa y 2 de penalización) y de canales (correo y SMS).

Ningún servicio crea sus colaboradores ni conoce clases concretas: pasar a SQL Server, SMTP o Twilio, o añadir WhatsApp, es escribir una clase que cumpla el contrato y cambiar el registro en Program.cs.

Copago. CalculadorCopago parte de 100 y aplica por Orden las reglas que corresponden: especialidad (10), convenio (20), primera vez (30) y radiografía (40). **Penalización.** CalculadorPenalizacion suma el valor de las reglas que aplican.

7.5. Cómo se resuelve cada violación SOLID

Principio	Problema en el legado (sección 4)	Solución en DentalCare.Refactored
SRP	GestorCitasOdontologicas reúne ocho responsabilidades de actores distintos.	Cada responsabilidad vive en su propia clase. Los servicios solo orquestan su caso de uso: ServicioAgendamiento tiene un único método público (Agendar) de 17 líneas.
OCP	Cadenas if/else por especialidad, convenio y penalización.	Las variantes son reglas (IReglaTarifa, IReglaPenalizacion). Una especialidad o un convenio nuevo se agrega con una clase y su registro en Program.cs, sin tocar calculadores ni servicios. Un canal de notificación nuevo se agrega como otra implementación de ICanalNotificacion.
LSP	Códigos primitivos en lugar de tipos intercambiables; Exception genérica.	Reglas, repositorios y notificadores son sustituibles a través de su contrato y los consumidores nunca preguntan por el tipo concreto. Las clases se declaran sealed para evitar jerarquías frágiles y el error de disponibilidad tiene su propio tipo.
ISP	EnviarEmailYSms y SqlServerEjecutor como contratos amplios.	Contratos de uno a tres miembros. La persistencia se divide en ICitaRepositorio e ICancelacionRepositorio, y los reportes en IRegistroReportes (escritura) e IConsultaReportes (lectura). ServicioCancelacion no conoce Guardar ni las consultas de totales. El envío por correo y por SMS, que en el legado era un solo método, queda en dos canales independientes.
DIP	El gestor crea con new a SqlServerEjecutor y NotificacionServicio; hay credenciales incrustadas.	Los servicios reciben abstracciones por constructor. El código de negocio deja de depender de System.Data.SqlClient y System.Net.Mail, y las credenciales desaparecen del código.

7.6. Verificación funcional

Compila sobre .NET 10; Program.cs ejecuta el escenario de la sección 3 y otro con convenio Prepagada sin primera visita ni radiografía. Salida de dotnet run (sin líneas de correo y SMS):

Agendada (EPS): \$ 130,00 · Cancelada: penalización \$ 90,00 · Agendada (Prepagada): \$ 25,00 · Total recaudado: \$ 155,00 · Total canceladas: 1

Los valores de aceptación (130,00 y 90,00) coinciden con la línea base; el total recaudado (155,00) no se descuenta al cancelar, igual que en el legado (hallazgo 4). El caso Prepagada ($25,00 = 250,00 \times 0,10$) también coincide y cada regla se comparó por separado.

Regla	Legado	Refactorizado	¿Equivale?
Factores por especialidad ($\times 1,2$; $\times 1,8$; $\times 2,5$; $\times 1,1$)	L.22–37	Tarifa* con Orden 10	Sí
EPS: el paciente paga el 30 %	L.40–43: copago $\times 0,30$	DescuentoEps: valor $\times 0,30$	Sí
Prepagada: el paciente paga el 10 %	L.44–47: copago $\times 0,10$	DescuentoPrepagada: valor $\times 0,10$	Sí
Primera vez (+20) y radiografía (+35)	L.49–57	RecargoPrimeraVez, RecargoRadiografia	Sí
Cancelación con menos de 24 h (50)	L.79–81	PenalizacionCancelacionTardia	Sí
Recargo de cancelación en Cirugía (+40)	L.81–82	RecargoCancelacionTardiaCirugia	Sí

8. Fase 3 — Recálculo de métricas e impacto cuantitativo

Como el gestor ya no existe, se comparan sus dos servicios sucesores, las demás clases con estado y la solución con sus paquetes (nivel para el que Martin, 2003, definió I, A y D), con los criterios de 6.1.

8.1. Criterios adicionales de medición

- **m:** métodos y propiedades con lógica o que implementan un miembro de interfaz; **a:** campos de instancia, sin constantes. Las clases sin estado se reportan «no aplica» en LCOM96b.

8.2. LCOM96b

Clase	a y m	Desarrollo	LCOM96b
ServicioAgendamiento	a = 7; m = 1 (Agendar)	Los 7 campos se usan en Agendar: $\mu = 1$ y $(1 - 1) / 1 = 0$ para cada uno. $(1 / 7) \times 0$	0,00
ServicioCancelacion	a = 5; m = 1 (Cancelar)	Los 5 campos se usan en Cancelar: $(1 / 5) \times 0$	0,00
CalculadorCopago	a = 1 (_reglas); m = 1	$\mu = 1 \rightarrow (1 - 1) / 1 = 0$	0,00
CalculadorPenalizacion	a = 1 (_reglas); m = 1	$\mu = 1 \rightarrow (1 - 1) / 1 = 0$	0,00
NotificadorCompuesto	a = 1 (_canales); m = 1	Notificar usa _canales: $\mu = 1 \rightarrow (1 - 1) / 1 = 0$	0,00
ReporteCitasEnMemoria	a = 2; m = 4	_totalRecaudado: $\mu = 2 \rightarrow 0,50$. _totalCanceladas: $\mu = 2 \rightarrow 0,50$. $(1 / 2) \times 1,00$	0,50
RepositorioEnMemoria	a = 2; m = 4	_citas: $\mu = 2 \rightarrow 0,50$. _cancelaciones: $\mu = 2 \rightarrow 0,50$. $(1 / 2) \times 1,00$	0,50

Clase	a y m	Desarrollo	LCOM96b
Promedio (7 clases con estado)	—	$(0 + 0 + 0 + 0 + 0 + 0,50 + 0,50) / 7$	0,14

Interpretación. La orquestación pasó de cohesión 0,50 a dos clases con 0,00. Las dos que siguen en 0,50 son adaptadores en memoria con dos interfaces cada uno; por ISP cada cliente ve solo la mitad que necesita.

8.3. Acoplamiento, inestabilidad, abstracción y distancia por clase

ServicioAgendamiento: $I = 10 / (10 + 1) = 0,909$ $A = 0$ $|D| = |0,00 + 0,909 - 1| = 0,091$

ServicioCancelacion: $I = 8 / (8 + 1) = 0,889$ $A = 0$ $|D| = |0,00 + 0,889 - 1| = 0,111$

Tipo	Ca	Ce	I	A	D	Observación
GestorCitasOdontologicas (legado)	1	5	0,83	0,00	0,17	Cinco dependencias concretas; dos de infraestructura.
ServicioAgendamiento	1	10	0,91	0,00	0,09	8 de 10 dependencias son interfaces; ninguna de infraestructura.
ServicioCancelacion	1	8	0,89	0,00	0,11	6 de 8 dependencias son interfaces.
CalculadorCopago	1	3	0,75	0,00	0,25	ICalculadorCopago, IReglaTarifa y SolicitudAgendamiento.
CalculadorPenalizacion	1	3	0,75	0,00	0,25	ICalculadorPenalizacion, IReglaPenalizacion y ContextoCancelacion.
RepositorioEnMemoria	1	3	0,75	0,00	0,25	ICitaRepositorio, ICancelacionRepositorio y Cita.
NotificadorCompuesto	1	3	0,75	0,00	0,25	INotificador, ICanalNotificacion y MensajeNotificacion.
Reglas de tarifa (8)	1	2–3	0,67–0,75	0,00	0,25–0,33	IReglaTarifa, SolicitudAgendamiento y, en seis de ellas, una enumeración.
IReglaTarifa (interfaz)	10	1	0,09	1,00	0,09	Abstracta y estable: muy cerca del extremo ideal ($A = 1$, $I = 0$).
ICitaRepositorio (interfaz)	2	1	0,33	1,00	0,33	Abstracción estable de la que depende el caso de uso.

Lectura del acoplamiento eferente. Que Ce suba (de 5 a 10 y 8) no es un retroceso: importa hacia dónde apunta. Las dependencias del legado eran clases concretas, dos atadas a SQL Server, SMTP y Twilio; ahora son interfaces el 80 % y el 75 %, y el resto, modelos de datos. Se cumple el Principio de Dependencias Estables (Martin, 2017): componentes volátiles ($I \approx 0,9$) que dependen de abstracciones estables ($I = 0,09$ y $0,33$). **Ce hacia infraestructura** baja de 2 a 0.

8.4. Métricas por paquete (namespace)

El legado vive en un único namespace, por lo que solo se analiza por clase; en la versión refactorizada cada carpeta es un namespace.

Paquete	Ca	Ce	I	A	D	Observación
Interfaces	4	2	0,33	1,00	0,33	Los 15 tipos son abstractos: paquete estable y abstracto.
Servicios	1	4	0,80	0,00	0,20	Concreto y volátil, cerca de la secuencia principal.
Reglas.Tarifas	1	3	0,75	0,00	0,25	Sus clases pueden cambiar sin afectar a nadie salvo a Program.cs.
Reglas.Penalizaciones	1	4	0,80	0,00	0,20	Mismo caso que las tarifas.
Modelos	5	2	0,29	0,00	0,71	Entidades y records muy usados: tienden a la Zona de Dolor, aceptable por su baja volatilidad.
Entidades	4	1	0,20	0,00	0,80	Zona de Dolor, igual que en el legado; son datos estables.
Enumeraciones	5	0	0,00	0,00	1,00	Zona de Dolor inofensiva: catálogos que casi no cambian.
Reglas	1	0	0,00	0,00	1,00	Solo contiene la constante PolíticaCancelacion.HorasLimite.
Excepciones	1	0	0,00	0,00	1,00	Un solo tipo, usado por el validador.
Program (composition root)	0	7	1,00	0,00	0,00	Máximamente inestable, como corresponde al punto de entrada.

Los paquetes volátiles (Servicios y Reglas) dependen de los estables, nunca al revés. Interfaces, único paquete abstracto, queda a 0,33 de la secuencia principal.

8.5. Abstracción de la solución

Legado: $A = 0 / 6 \text{ tipos} = 0,00$ (por miembros: $0 / 8 = 0,00$)

Refactorizado: $A = 15 / 48 \text{ tipos} = 0,31$ (por miembros: $21 / 70 = 0,30$)

Los 48 tipos son 15 interfaces y 33 concretos; por miembros, 21 abstractos frente a 49 concretos.

8.6. Complejidad ciclomática

Método	Puntos de decisión	Complejidad
ServicioAgendamiento.Agendar	Sin decisiones (flujo lineal)	1
ServicioCancelacion.Cancelar	Sin decisiones (flujo lineal)	1
CalculadorCopago.Calcular	foreach (L.24) e if (L.26)	3
CalculadorPenalizacion.Calcular	Sin decisiones (filtrado declarativo con LINQ)	1
NotificadorCompuesto.Notificar	foreach sobre los canales	2
ValidadorDisponibilidadPorBandera.Validar	if (L.15)	2

Método	Puntos de decisión	Complejidad
RecargoCancelacionTardiaCirugia.Aplica	&& (L.13)	2
Demás reglas (Orden, Aplica, Aplicar)	Sin decisiones	1

La complejidad máxima baja de 10 (AgendarCita) a 3: las ramas if se volvieron clases y cada pieza se prueba por separado.

8.7. Tabla comparativa Antes vs. Después

Métrica	Antes (Legacy)	Después (Refactored)	Lectura
LCOM96b del orquestador	0,50	0,00 en ambos servicios; 0,14 de promedio en clases con estado	Cada clase tiene una sola razón de cambio.
Ca	1	1 por servicio	Sin cambio: el único cliente sigue siendo Program.cs.
Ce	5 (todas concretas)	10 y 8	Crece en número, pero ahora apunta a abstracciones estables.
Dependencias que son interfaces	0 %	80 % y 75 %	El resto son entidades y modelos encapsulados.
Ce hacia infraestructura	2 (SqlServerEjecutor, NotificacionServicio)	0	El negocio ya no conoce SQL Server, SMTP ni Twilio.
I	0,833	0,909 y 0,889	Sigue alta, como corresponde a un caso de uso; ahora depende de componentes más estables.
A por clase	0,00	0,00	Una clase concreta siempre tiene A = 0; la abstracción aparece en la solución.
A de la solución (por tipos)	0,00 (0 / 6)	0,31 (15 / 48)	15 interfaces frente a 33 tipos concretos.
A de la solución (por miembros)	0,00 (0 / 8)	0,30 (21 / 70)	Confirma el resultado por tipos.
D	0,167	0,091 y 0,111	Los servicios quedan algo más cerca de la secuencia principal.
Complejidad ciclomática máxima	10	3	Desaparecen las cadenas if/else.

Métrica	Antes (Legacy)	Después (Refactored)	Lectura
Método más largo	62 líneas (AgendarCita)	17 líneas (Agendar)	Métodos cortos y legibles.
Tipos concretos / abstractos	6 / 0	33 / 15	Más piezas, cada una pequeña y con un propósito claro.
Violaciones SOLID	5 (3 directas y 2 indirectas)	0 (ver 7.5)	Los cinco principios quedan atendidos.

El impacto está en la cohesión (0,50 → 0,00), el tipo de acoplamiento (de concretos e infraestructura a interfaces), las abstracciones (A de 0,00 a 0,31) y la complejidad (10 → 3); D del gestor ya era baja.

8.8. Impacto en atributos de calidad

Atributo	Legado	Refactorizado
Modificabilidad	Cambiar una tarifa obliga a editar AgendarCita y volver a probar todo el flujo.	Se edita una clase de unas 15 líneas sin tocar el resto.
Extensibilidad	Una especialidad nueva exige otra rama if/else en AgendarCita.	Una clase nueva que implemente IReglaTarifa, un valor en la enumeración y su registro en Program.cs.
Testabilidad	Es imposible probar AgendarCita sin SQL Server ni servidor de correo.	Cada regla y cada servicio se prueban con dobles en memoria, como los que ya trae la solución.
Seguridad	Contraseña de SQL Server y clave de Twilio en el código fuente.	No hay credenciales en el código; la configuración será responsabilidad del adaptador real.
Reemplazabilidad	Proveedores fijos (SQL Server, SMTP, Twilio).	Cambiar de proveedor es escribir un adaptador y modificar una línea de Program.cs.
Legibilidad	Un método de 62 líneas con 9 decisiones.	Métodos de 1 a 17 líneas con nombres del dominio.

9. Fase 4 — Decisiones arquitectónicas (ADRs)

Las decisiones se documentan en registros Markdown en docs/adr/, con el formato de Nygard visto en clase (Richards y Ford, 2020) y alineados con ISO/IEC/IEEE 42010:2022.

Registro	Decisión	Fuerzas principales	Trade-offs aceptados	Verificación de cumplimiento
ADR-001: Inversión de dependencias para persistencia y	Los servicios dependen de interfaces abstractas (ICitaRepositorio, ICancelacionRepositorio, INotificador)	Testabilidad unitaria , reemplazabilidad de proveedores de	Incremento en el número de clases, mayor acoplamiento o eferente en	Ausencia de instanciaciones directas (new) fuera de Program.cs y aislamiento total del paquete de dominio

notificaciones (DIP)	y reciben sus implementaciones por constructor. INotificador se implementa mediante un patrón compuesto que delega en canales independientes de correo y SMS.	infraestructura y retiro de credenciales incrustadas en el código.	servicios y la necesidad de mantener un Composition Root explícito.	respecto a librerías de infraestructura.
ADR-002: Políticas de cancelación y tarifas mediante Estrategia y Compuesto (OCP)	Encapsulamiento de cada tarifa, descuento, recargo y penalización como una regla independiente (IReglaTarifa, IReglaPenalizacion) coordinada por un calculador compuesto.	Modificabilidad y extensibilidad de las políticas financieras sin alterar la lógica central existente.	Dependencia del orden de ejecución de las reglas y mayor número de clases pequeñas en el proyecto.	Verificación mediante pruebas unitarias por regla y cumplimiento de los casos de prueba de integración funcional.
ADR-003: Sustitución de cambios por interfaces puras (LSP)	Aplicación estricta de contratos intercambiables. Todo comportamiento variante implementa interfaces puras y sustituibles sin requerir condicionales de tipo en los orquestadores.	Polimorfismo puro , comportamiento determinista y eliminación del acoplamiento a clases concretas en el flujo principal.	Mayor exigencia en el diseño de abstracciones y resolución interna de configuraciones específicas en cada adaptador.	Prohibición de verificaciones de tipo (is/as) y garantía de que ninguna clase implementadora lance NotImplementedException.
ADR-004: Asignación de responsabilidades únicas por componente (SRP)	Descomposición de la clase monolítica en componentes especializados por rol: servicios orquestadores, calculadores, reglas financieras y adaptadores.	Alta cohesión en los servicios , mejor legibilidad y aislamiento de motivos de cambio (clínica, finanzas y tecnología).	Incremento significativo en la cantidad de clases y un flujo de ejecución indirecto a través de componentes especializados.	Revisiones de código para asegurar que ningún componente asuma responsabilidades cruzadas fuera de su dominio.
ADR-005: Segregación	Fraccionamiento de interfaces	Mínimo acoplamiento	Multiplicación de	Auditorías de diseño para confirmar la

de contratos específicos por cliente (ISP)	monolíticas en contratos de rol granulares y específicos según la necesidad de cada consumidor.	o, eliminación de dependencias sobre métodos no utilizados por el cliente y simplificación de dobles de prueba.	archivos de interfaz y necesidad de que las clases concretas implementen múltiples contratos.	ausencia de métodos no consumidos o innecesarios en los contratos de cada cliente.
---	---	--	---	--

10. Conclusiones

GestorCitasOdontologicas era una God Class: violaba SRP, OCP y DIP de forma directa y LSP e ISP por ausencia de abstracciones; su D = 0,167 ocultaba el problema, visible en la mezcla dentro de AgendarCita y la complejidad de 10.

La refactorización la reemplaza por componentes cohesivos y sustituibles: cohesión perfecta en los orquestadores, 31 % de tipos abstractos, dependencias hacia componentes estables, complejidad máxima reducida a un tercio y equivalencia funcional comprobada (130,00 y 90,00). Las decisiones quedan en los ADRs.

11. Referencias

- Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice* (4th ed.). Addison-Wesley.
- Henderson-Sellers, B. (1996). *Object-Oriented Metrics: Measures of Complexity*. Prentice Hall.
- Institución Universitaria ITM (2024). *Microdiseño Curricular: Arquitectura de Software I (FDE 058)*. Facultad de Ingenierías, Medellín.
- ISO/IEC/IEEE (2022). *ISO/IEC/IEEE 42010:2022 Software, systems and enterprise — Architecture description*.
- Martin, R. C. (2003). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall.
- Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
- Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media.
- Seemann, M., & van Deursen, S. (2019). *Dependency Injection: Principles, Practices, and Patterns*. Manning.
- Material de clase, Arquitectura de Software I: *principios SOLID (Módulos I y II), Clase 14 (ADRs) y Clase 15 (Diseño sostenible y métricas)*.